String width library Writeup **ODDITY X home assignment Elad Elya**

Start with some research about the problem - I realized the main problem is that the UNICODE WIDTHS TABLE is huge, and the main goal is to find a better way to calculate string width than loading the whole table.

Also a single emoji of a family for example might be many chars with another joiner (with zero width) chars, and to analyze the width a decomposition is needed.

More I checked about the structure of a UNICODE string, and I learned that to break a string to graphemes, *VERY GENERALLY* I need to look for each char if the next char is a zero width joiner. If so, the next char is still part of the current grapheme. Else, the current grapheme has ended and now we can move on to analysing its length. Looks like there is a tool that does so - Intl.Segmenter. Making a custom one looks complicated because of many special cases (it's not really just if there is a ZWJ then continue, else end of grapheme) and redundant. It might be considered so the library can run in older versions of JS that don't have this implementation.

So far, I saw that generally Emojis are 2 units width, and non Asian letters are 1 unit width, and there is a standard named East Asian Width that shows which East Asian letter are 2 units width and which aren't, and they are ordered comfortably in a sequence of same width (All 2-width letters are in specific ranges, and not distributed randomly).

Another thing that should be taken into account is that strings might have design codes (like text color) and those codes need to be ignored (ANSI / Escape Codes). There are tools to handle that - strip-ansi for example. Making a custom one probably is not that bad, we'll see later what's more efficient.

Also there are edge cases, letters that have different widths on different terminals, need to see if possible to check the terminal type or something and know at runtime how the grapheme should be handled (after a check it looks like there is no such tool). Also need to think about options that need to be added, like if the user wants to ignore \n or such.

—----------

After understanding the problem, I can move on to solving it.
So loading the whole Unicode Widths Table is not the solution, although it's the easiest approach. It will make the library way too big.

So when we get a string, we first want to clean the ANSI part of the string, since they don't "cost" width. So we will need a strip ansi function (strip-ansi or custom).

Then, separate the string to its graphemes (Intl.Segmenter or custom).

And now the core of the code: calculating width for each grapheme.
We iterate over the grapheme components:

1. If it's a ZWJ (zero width joiner), or such, return 0.

Create a set of all these characters and check if the current char is in it at O(1).

2. If an emoji is involved, then the terminal will use 2 width units to print the whole grapheme, unless a vs15 (force 1 width) unicode is part of the grapheme.

Save list of ranges of emojis, as will be explained in the next lines. O(log(k)) where k is the ranges amount. I know there is a method that checks emojiness with regular expressions, but using the list is very efficient since the log of the number of ranges will be basically O(constant), and it's more consistent with the next part architecture. Also emojis length can vary depending on the chars that come after them in the grapheme (VS15, VS16). So when we will get to the code of this part a further explanation will be.

3. Then East Asian check, look at the East Asian Table and see whether it's a wide char or not.

Note that no need to save the whole actual table, just the ranges of unicodes that are wide, and we save them ordered in increasing order, so we can lookup at the table for a specific value in O(log(k)) iterations (where k in the amount of ranges) with simple binary search.

4. If none of these, it's a regular 1-width char and return 1 as the width.

Note that all the implementations mentioned are just the general idea and in the middle of writing the code probably many things will change :)

—----------

Implementing:
Starting with the Emojis and East Asian ranges:
The Unicode table can be changed. Thus, the library should have an engine that calculates the ranges at build time, and maybe add an option that lets the user give their own ranges if they don't want/can't rebuild. This option should be marked as not recommended if I'll add it eventually (Elad from the future say better option is that the user will manually supply the updated EastAsianWidth.txt and emoji-data.txt original unicode files).
For the first version, it's better to hardcode the ranges.
From unicode.org we can get the ranges for now:
The whole list of emojis:
https://www.unicode.org/emoji/charts/full-emoji-list.html
List of Unicodes for basic and non complex emojis:
https://www.unicode.org/Public/UCD/latest/ucd/emoji/emoji-data.txt
List of Unicodes for East Asian Width:
https://www.unicode.org/Public/UCD/latest/ucd/EastAsianWidth.txt
(I treated the ambiguous width as one for now, an option that gives the user control of that is nice to add)
Libraries that can give us the current ranges can be found here:
https://cldr.unicode.org/#TOC-What-is-CLDR-
For now I'll hardcode the ranges.
The code is pretty straightforward.
ranges.ts has the hardcoded ranges, and the binary search function that checks whether a specific unicode is within one of the ranges.
index.ts just exports out the main function, stringWidth.
width.ts has the core logic.
The code has notes that explain it well.
utils.ts just gives the stripAnsi function.

Edge/Ignored cases

1. If a string has both vs15 and vs16 unicodes, it will act as it has width of 1.
2. If an emoji is involved in a grapheme, its width is 2 (unless there is a v15 unicode) without considering emojis that might have default width 1. (☀, ♣ for example).
3. Control characters such as `\t`, `\n`, `\r`, `\0`, and bidi control characters (RTL, LTR..) currently have width 1. Their handling should be made explicitly.
4. In case more emojis are added… the ranges are hardcoded as mentioned. Marked cause we took it into account the simplicity over correctness tradeoff to supply fast something the works well for almost all cases.
5. Unsupported graphemes, for example {english_letter}+{zwj}+{emoji} might be printed as two chars with total width 3, although the function for a single grapheme will return. That shouldn't be a problem since the Intl.Segmenter should split to proper graphemes.
6. Letter + vs16 will return 2, although it has width 1
   // fixed //
7. some combining marks outside the currently hardcoded zero-width ranges (like ◌) may return width 1 when used alone. When attached to a character, Intl.Segmenter groups them into the same grapheme.
   I just saw that "`\u0301\u0308`" has width 0, although it does take space on the screen without a character. So it's a bit inconsistent. Probably something wrong with the hardcoded ranges.
8. " 🇮🇱 " is a flag but has width 1. The program returns width 2.
   // fixed //
9. As mentioned, returns 1 for ambiguous-width east asian chars. Not necessarily a problem but worth mentioning.
10. Unpaired surrogate code units have width 1. Usually they are printed as a question mark of width 1 so it's ok, just nice to mention as well.

(Green marked sections are fixed or not really a problem just worth mentioning)

Changes to fix section 8:
So apparently flags are two Regional Indicator Symbols connected. So just one should not count as a regular emoji with width 2.
So need to modify the EMOJI_RANGES in ranges.ts and remove `{ start: 0x1F1E6, end: 0x1F1FF }, // Regional Indicator Symbols (Flags)`
And take care of flags specifically.
Add a function that checks wether a unicode is a regional indicator:

```
export function isRegionalIndicator(codePoint: number): boolean {
    return codePoint >= 0x1F1E6 && codePoint <= 0x1F1FF;
}
```

And keep track of regionalIndicatorCount in the grapheme processing.
/fixed/

I think at that point, after I made some tests, the first version can be released 🙂